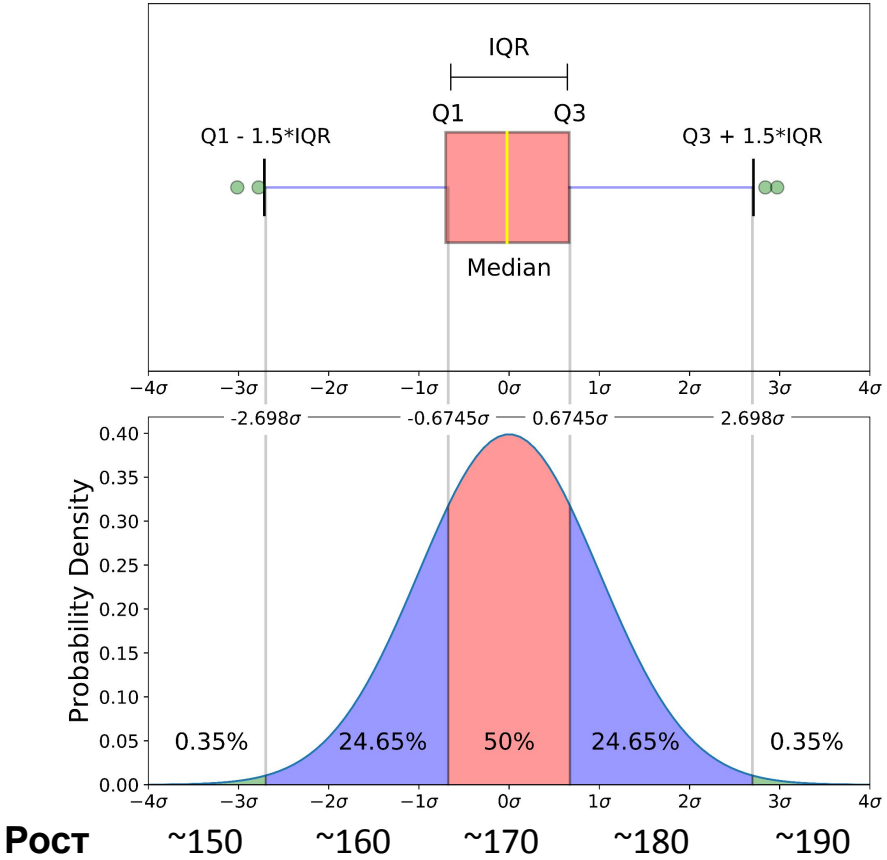
ИТМО

# Алгоритмы и структуры данных

## Лекция 8

Расширение структур данных

# Порядковые статистики



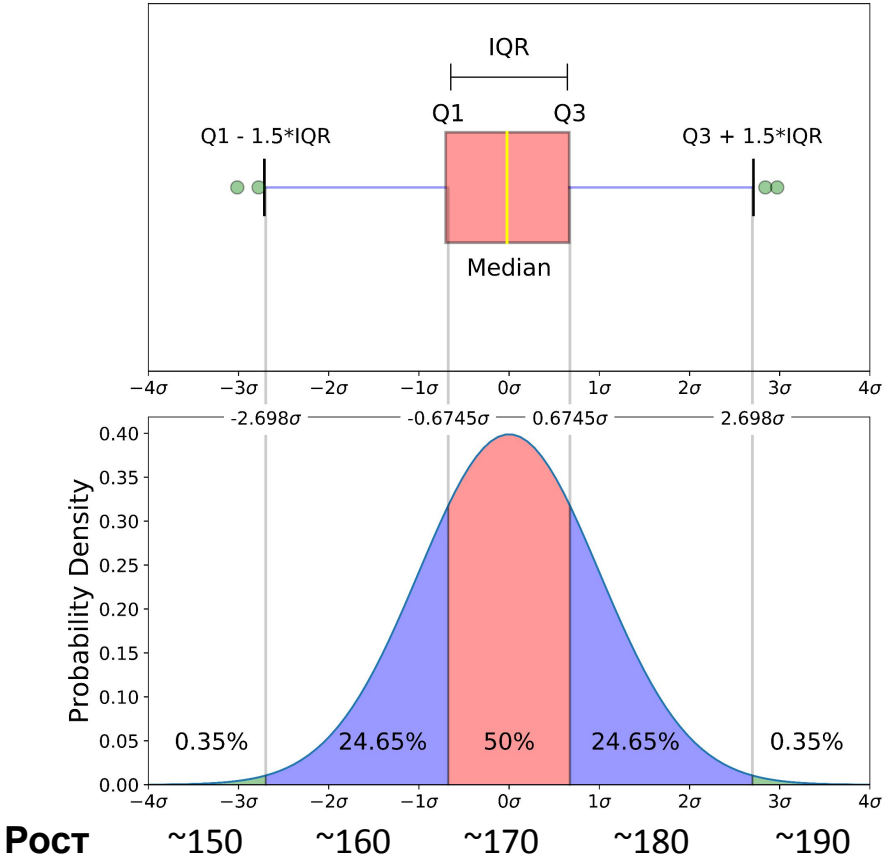

Что если мы построим <u>распределение роста</u>:

[160, 180, 165, 170, 168, 171, 175, 190, 166, …]

**И вот мы можем посчитать:**

- Медиану квантиль 50 - наиболее встречаемое
- Минимальный рост
- Максимальный рост
- Квантиль 97.5 - за которым гиганты (выбросы)
- Квантиль 2.5 - перед которым компактные
- Квантили 25 и 75 - наиболее встречаемые

# Например, чтобы посчитать медиану




Что если мы построим <u>распределение роста</u>:

[160, 180, 165, 170, 168, 171, 175, 190, 166, …]

Сортируем ряд

[160, 165, 166, 168, 170, 171, 175, 180, 190]

**Берем:**

- N // 2 + 1 значение, если N % 2 != 0
- AVG от [N // 2, N // 2 + 1], если N % 2 == 0

[160, 165, 166, 168, 170, 171, 175, 180, 190]

**Медиана**

# Представим кейс – Sentry

ИТМО

| TRANSACTION | PROJECT | TPM ↓ | P50 | P95 | FAILURE RATE |
|---|---|---|---|---|---|
| /api/services | gateway | 8,03/min | 12.00ms | 22.00ms | 0% |
| /api/services/ | gateway | 7,74/min | 19.00ms | 32.00ms | 0% |
| /api/graphs/size | graph-server | 4,06/min | 10.00ms | 17.76s | 0,8% |
| cvat.tasks.check_cvat_task_status | gateway | 3,98/min | 1.00s | 1.18s | 0% |
| /api/profile/current/ | gateway | 1,99/min | 25.00ms | 45.00ms | 11,18% |
| uploaded_models.tasks.synchronize_ ··· | gateway | 0,992/min | 172.00ms | 4.12s | 22,46% |

Квантили

# И вот есть задача

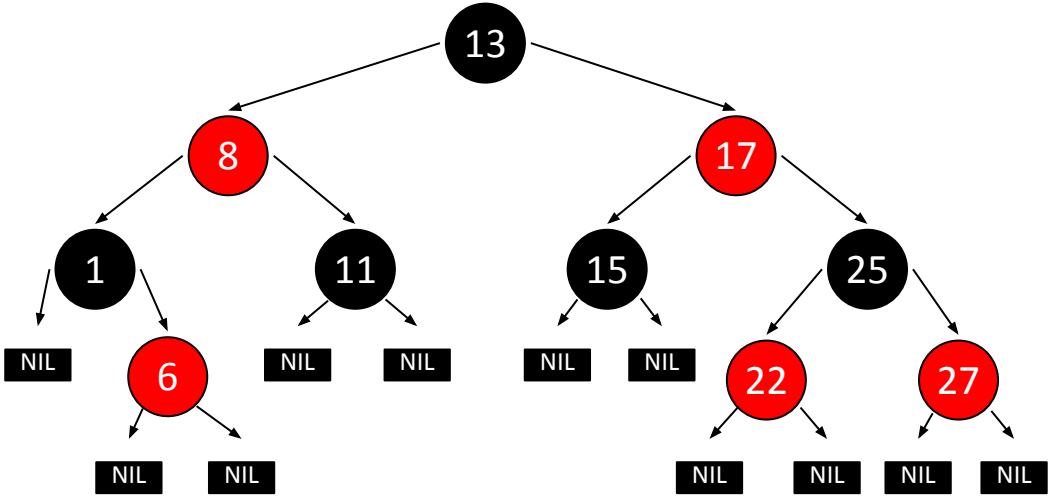
Есть множество значений {0, 1, …, N}

**Каждую секунду** множество изменяется:
добавление или удаление элементов

Нужно при каждом изменении пересчитывать порядковые статистики

# Вспоминаем красно-черные деревья

ИТМО

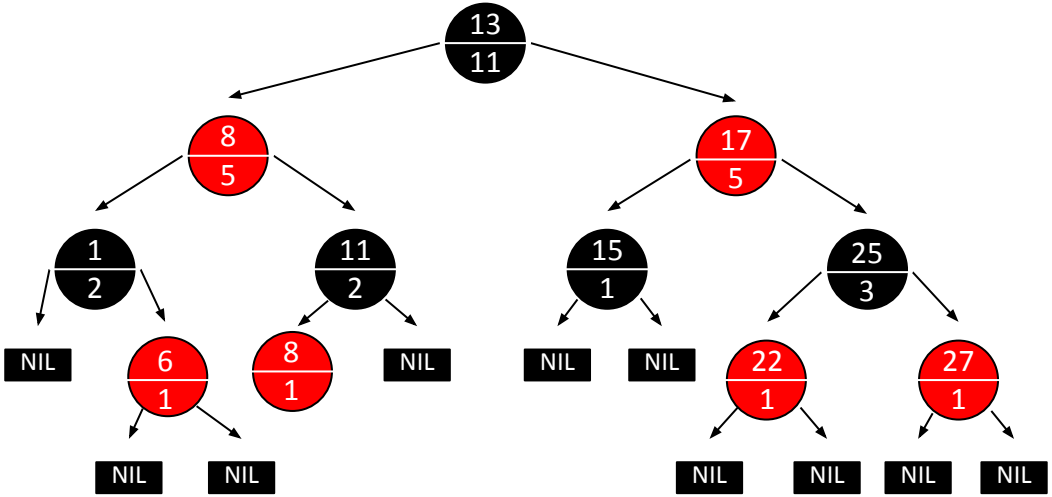



Приближенно сбалансированное дерево

**Свойства:**

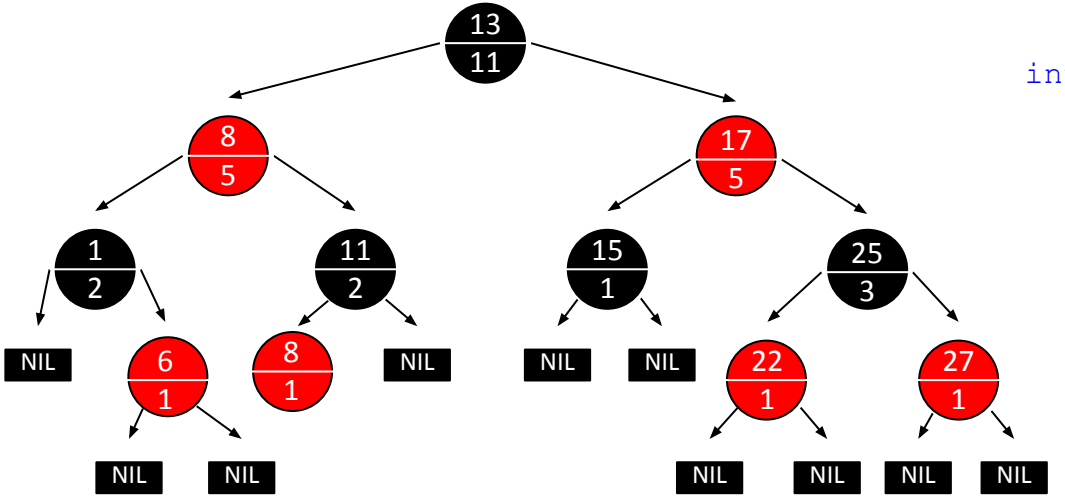
1. Узел - **красный** или **черный**.
2. Корень - **черный**
3. Лист (NIL) - **черный**
4. Узел **красный**, дети **черные**
5. Все простые пути от узле до листьев, содержат одинаковое количество **черных** узлов.

# Модифицируем структуру дерева




```
struct OrderStatNode {
    int key;
    char color;

    int size;

    OrderStatNode* parent;
    OrderStatNode* left;
    OrderStatNode* right;
}
```

# Получение индекса каждого узла

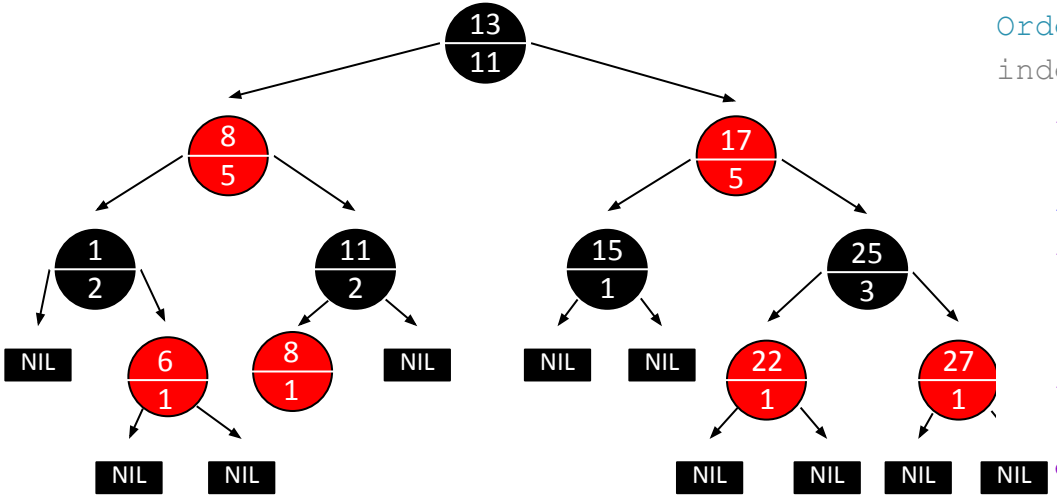



```
int get_rank(OrderStatNode* node) {
    int rank = 0;
    if (node->left != nullptr)
        rank = node->left->size;

    OrderStatNode* next = node;
    while (next != root) {
        if (next == next->parent->right) {
            if (next->parent->left != nullptr)
                rank +=
next->parent->left->size;
            rank += 1;
        }
        next = next->parent;
    }

    return rank;
}
```

**std::cout →**

node1 - 0
node6 - 1
node8.2 - 2
node8 - 3
node11 - 4
node17 - 7

# Как искать порядковые статистики

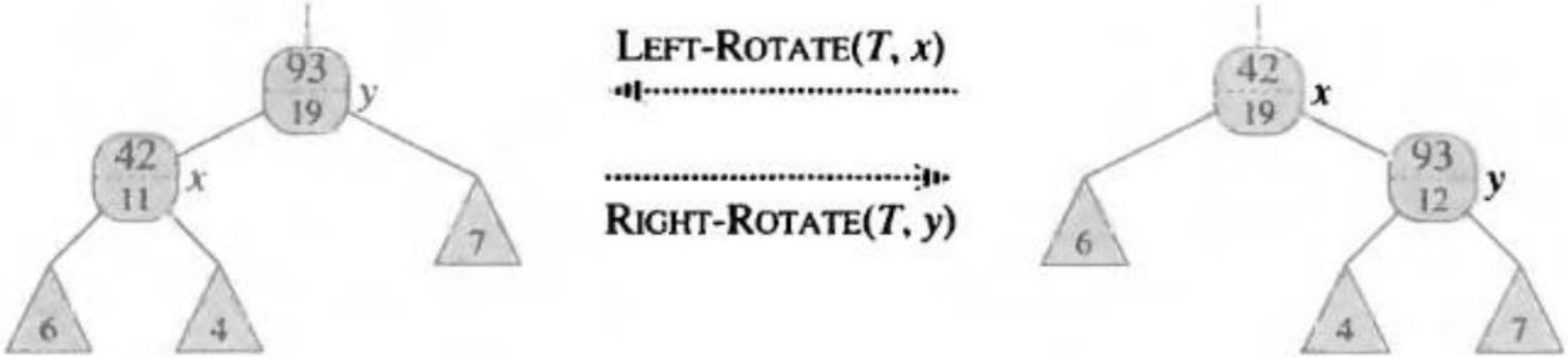



```
OrderStatNode* select(OrderStatNode* node, int
index) {
    if (node == nullptr) return 0;

    int rank = 0;
    if (node->left != nullptr)
        rank = node->left->size;

    if (rank == index)
        return node;
    else if (index < rank)
        return select(node->left, index);
    return select(node->right, index - rank);
}

OrderStatNode* quantile(float quantile) {
    int index = int(root->size * quantile);
    return select(this->root, index - 1);
}
```

**std::cout** →

0.05 - 1
0.50 - 13
0.95 - 27

# В вставке изменяется только операция поворота



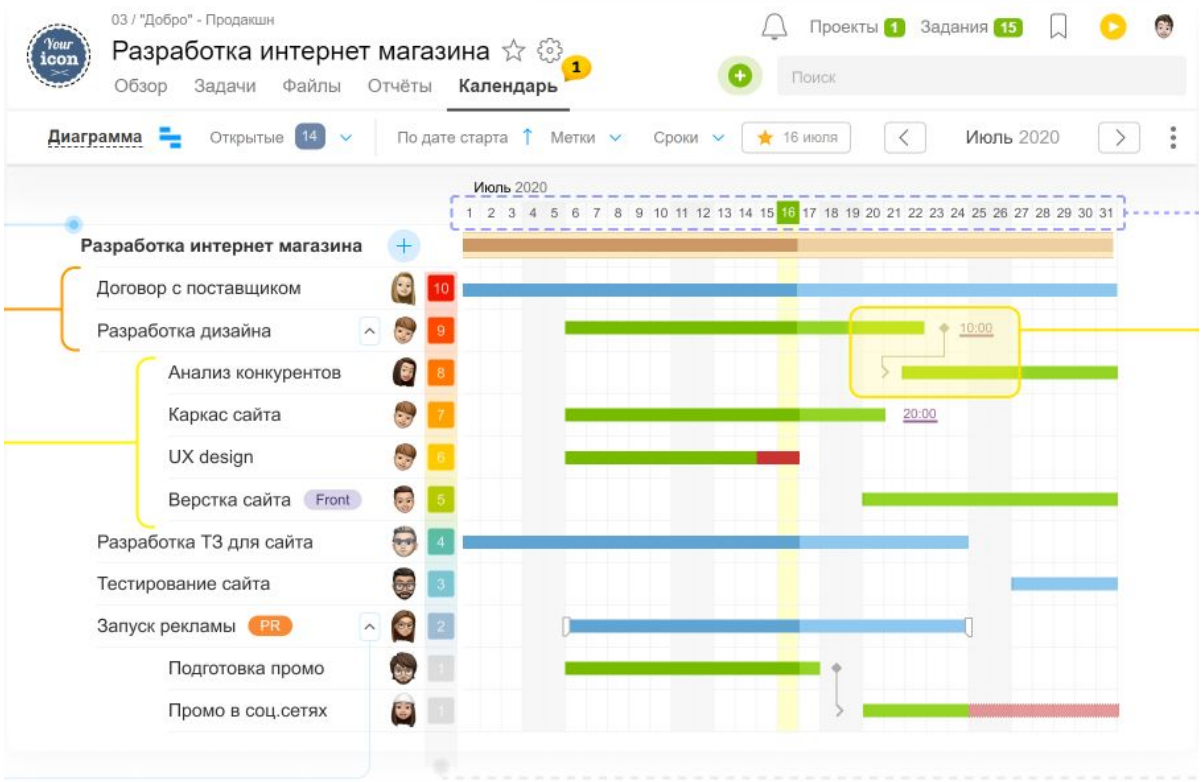
# Расширение структур данных

ИТМО

Расширение структур данных можно разбить на **четыре шага**.

1. Выбор базовой структуры данных.
2. Определение необходимой дополнительной информации, которую следует хранить в базовой структуре данных и актуальность которой следует поддерживать.
3. Проверка того, что дополнительная информация может поддерживаться основными модифицирующими операциями над базовой структурой данных.
4. Разработка новых операций

# Новая задача - диаграмма Ганта

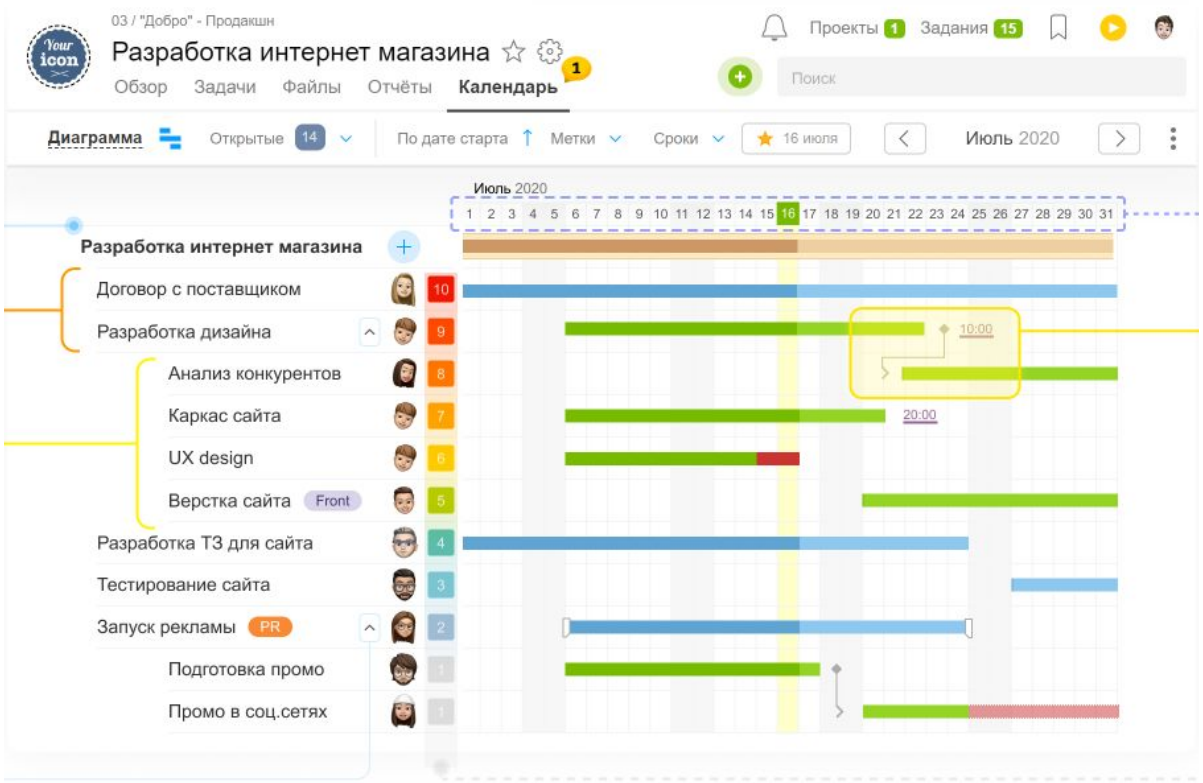



**Есть работы** (интервалы):

- начало работы (дата)
- конец работы (дата)

**Нужна:** <u>эффективная</u> структура данных, для хранения, обновления и фильтра работ

# Новая задача - диаграмма Ганта




**Есть работы** (интервалы):

- начало работы (дата)
- конец работы (дата)

```
[
  [1, 2],
  [3, 7],
  [5, 6],
  [8, 10],
  ...
]
```

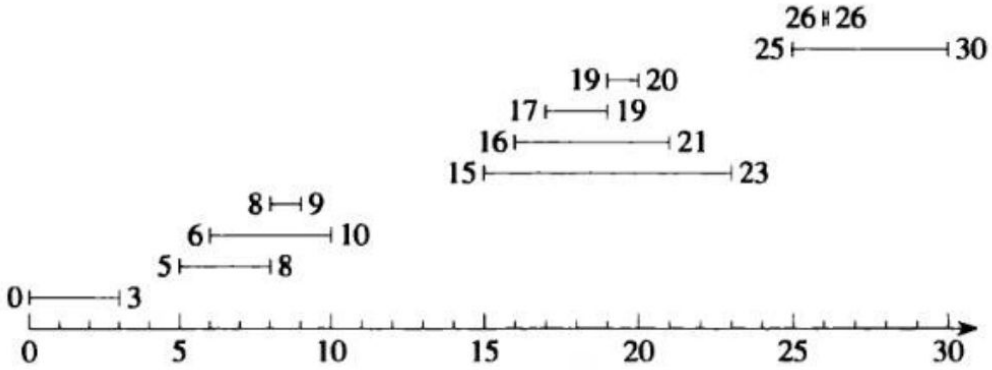
**Нужна:** <u>эффективная</u> структура данных, для хранения, обновления и фильтра работ

# Новая задача - диаграмма Ганта

**Есть работы** (интервалы):

- начало работы (дата)
- конец работы (дата)

```
[
    [1, 2],
    [3, 7],
    [5, 6],
    [8, 10],
    …
]
```




**Нужна:** <u>эффективная</u> структура данных, для хранения, обновления и фильтра работ

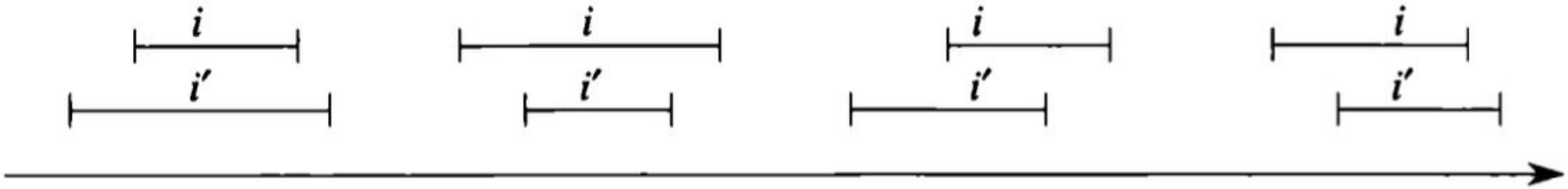
**Как решать?**

# Дерево отрезков (интервалов)

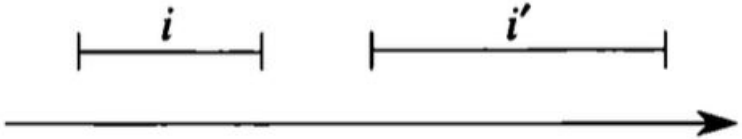
**Строится** по min
**Хранит** - max

$$x.max = \max(x.int.high, x.left.max, x.right.max)$$

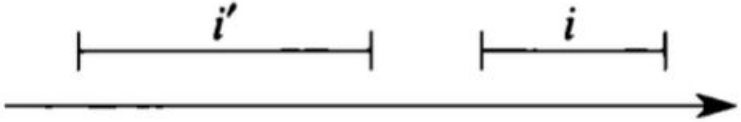
# Нужно договориться, что значит если отрезки “пересекаются”




Отрезки пересекаются

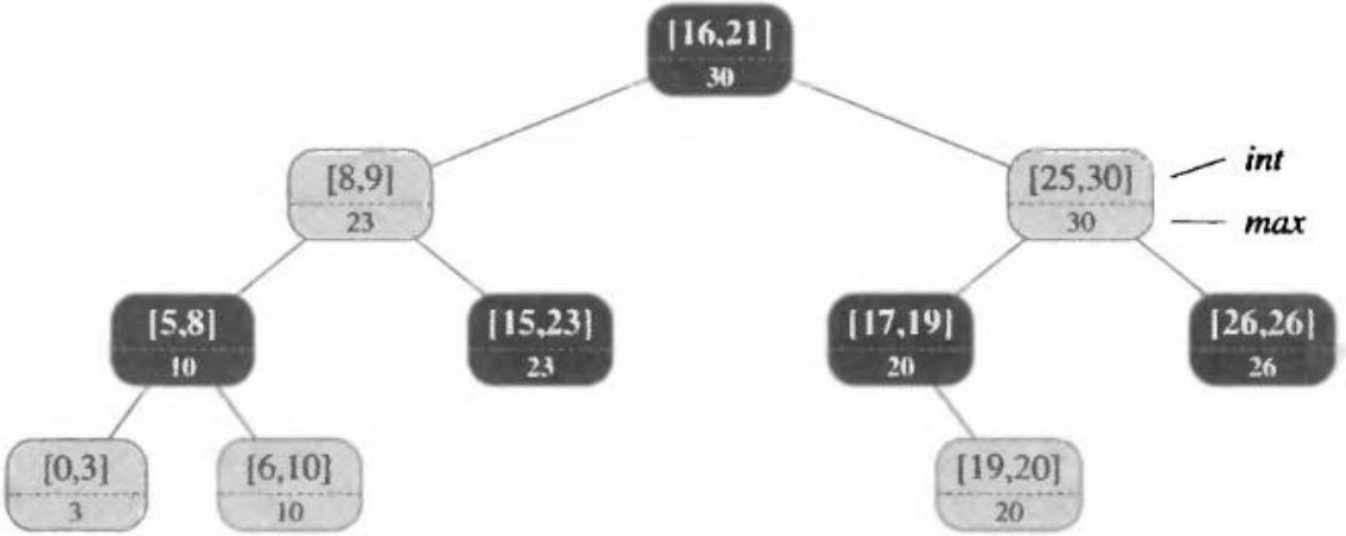



Отрезки НЕ пересекаются




Отрезки НЕ пересекаются

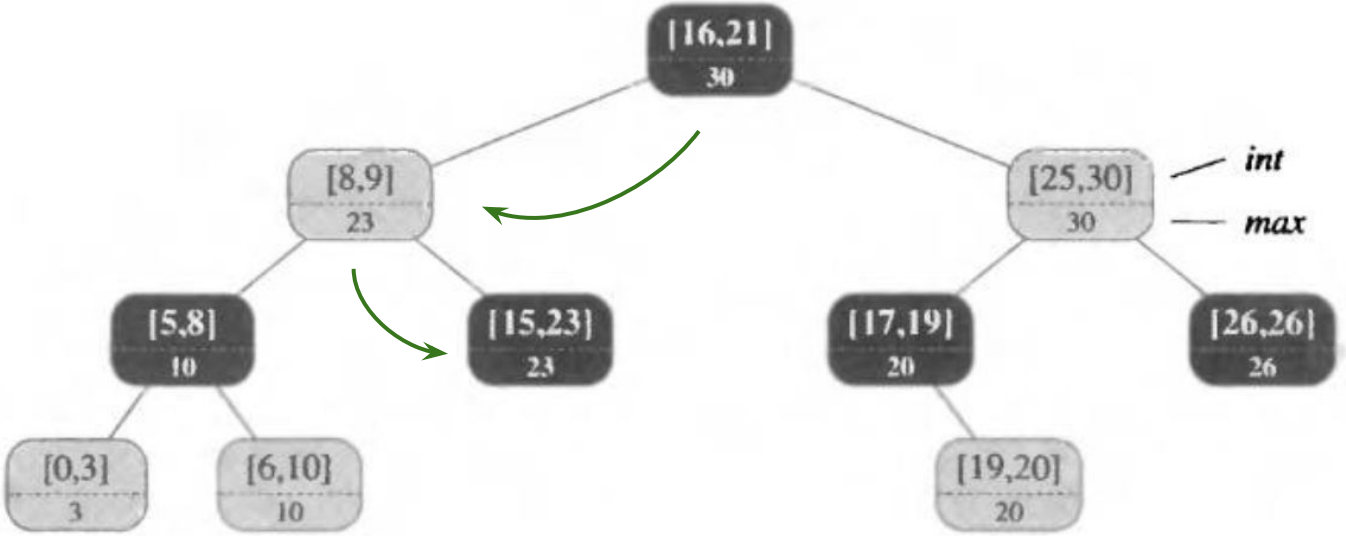
# Поиск пересекающегося отрезка




**Входной отрезок:**
[22, 24]

Найти <u>пересекающийся отрезок</u>

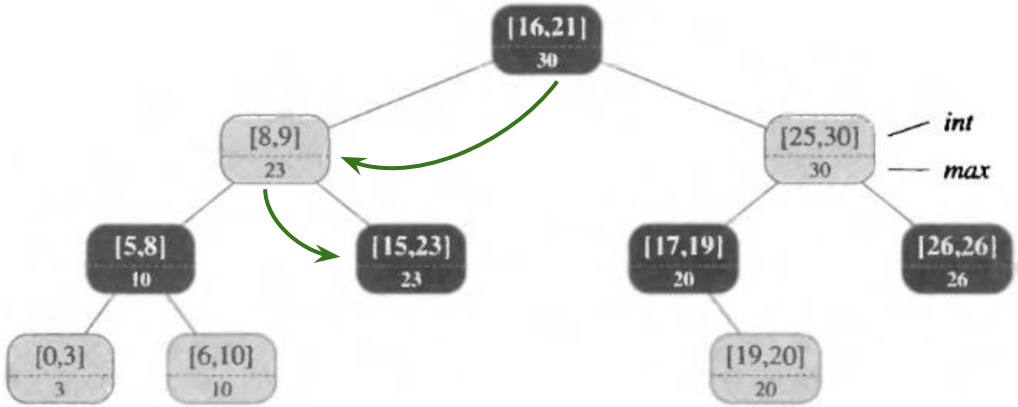
# Поиск пересекающегося отрезка (решение)




**Входной отрезок:**
[22, 24]

Найти <u>пересекающийся отрезок</u>

# Поиск пересекающегося отрезка (решение)



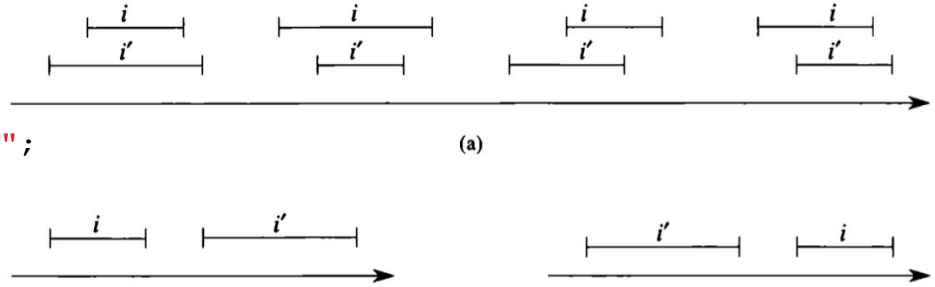

**Входной отрезок:**
[22, 24]

```
IntervalNode* interval_search(Interval interval) {

    IntervalNode* x = this->root;

    while (x != nullptr && !(interval & x->interval)) {

        bool is_left = x->left != nullptr;
        is_left &= x->left->max >= interval.min;

        x = is_left ? x->left : x->right;
    }
    return x;
}
```

# Листинг структуры Interval

ИТМО

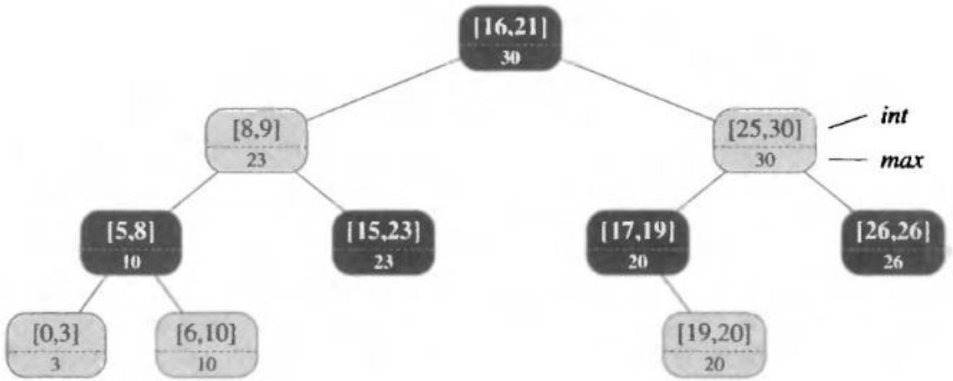
```
struct Interval {
    int min, max;
    std::string key;

    Interval(int min_, int max_) {
        if (min  >= max ) {
            throw std::invalid_argument("max must be more than min");
        }
        min = min, max = max_;
    }
    std::string str() const {
        std::ostringstream oss;
        oss << "[" << min << ", " << max << "]";
        return oss.str();
    }

    bool operator&(Interval i_) {
        Interval i = *this;
        bool intersect = i .min < i.min && i .max > i.max | i .min > i.min && i .max < i.max;
        intersect |= i_.min < i.min && i_.max > i.min | i_.min < i.max && i_.max > i.max;
        return intersect;
    }
};
```



# Листинг структуры IntervalNode

ИТМО

```
struct IntervalNode {
   char color;
   Interval interval;
   int max;  // max of the current and children from interval

   IntervalNode* parent, left, right;

   IntervalNode(char color_, Interval interval_) : color(color_), interval(interval_) {
       max = interval_.max;
   };

   ~IntervalNode() {
       delete left;
       delete right;
   }
}
```



# А теперь давайте думать

**Входной отрезок:**
[10, 20]

Найти
<u>бинарное дерево</u>

**Мне кажется**

**Или у вас есть вопросы?**

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru